

Infrastructure for BERT and GPT

Optimizing Training & Serving of Foundational Transformers

A technical overview for ML engineers and infrastructure architects
building production-ready transformer pipelines.



Why BERT & GPT Matter

The foundational transformer architectures powering today's LLM ecosystem each bring distinct infrastructure requirements:

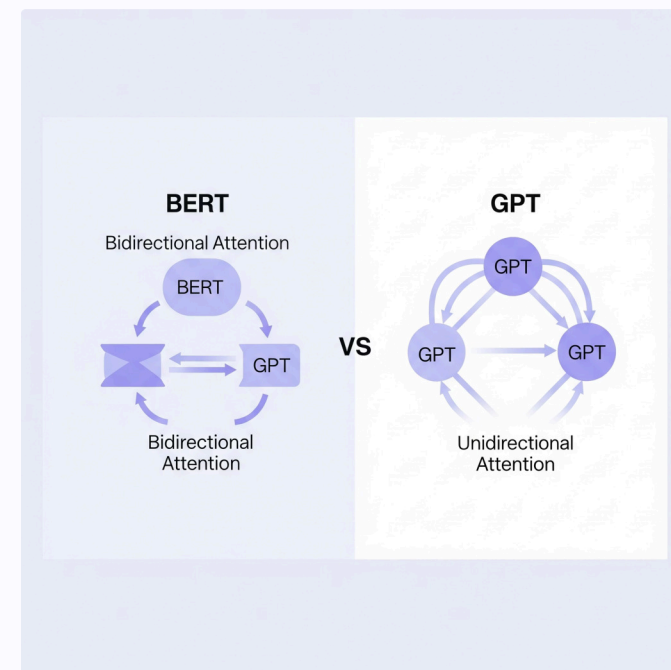
BERT

Pioneered **bidirectional pretraining** for language understanding tasks

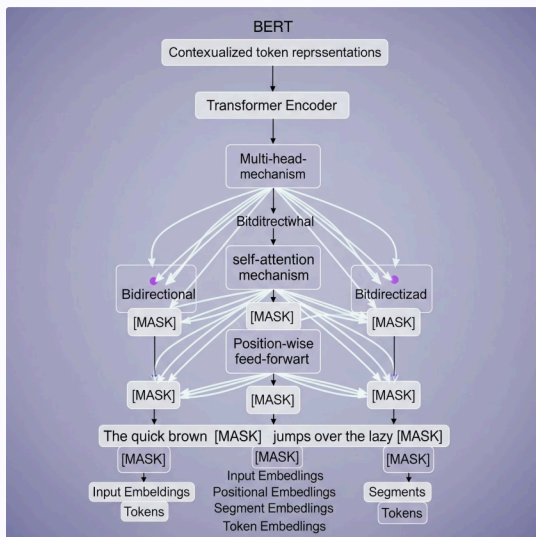
GPT

Popularized **autoregressive generation** for text completion and creative tasks

Both architectures require robust infrastructure for [training](#), [fine-tuning](#), and [serving](#) at scale.



BERT Infrastructure Needs



1

Encoder-Only Training

Optimized for **masked language modeling** with bidirectional context

2

Data Scale

Pretraining on terabyte-scale text corpora with sophisticated masking strategies

3

Downstream Tasks

Fine-tuning architecture for QA, classification, NER, and other understanding tasks

4

Serving Requirements

Need **fast embedding serving** for production applications with strict latency constraints

GPT Infrastructure Needs

Decoder-Only Architecture

Optimized for **causal language modeling** with unidirectional attention

Memory Challenges

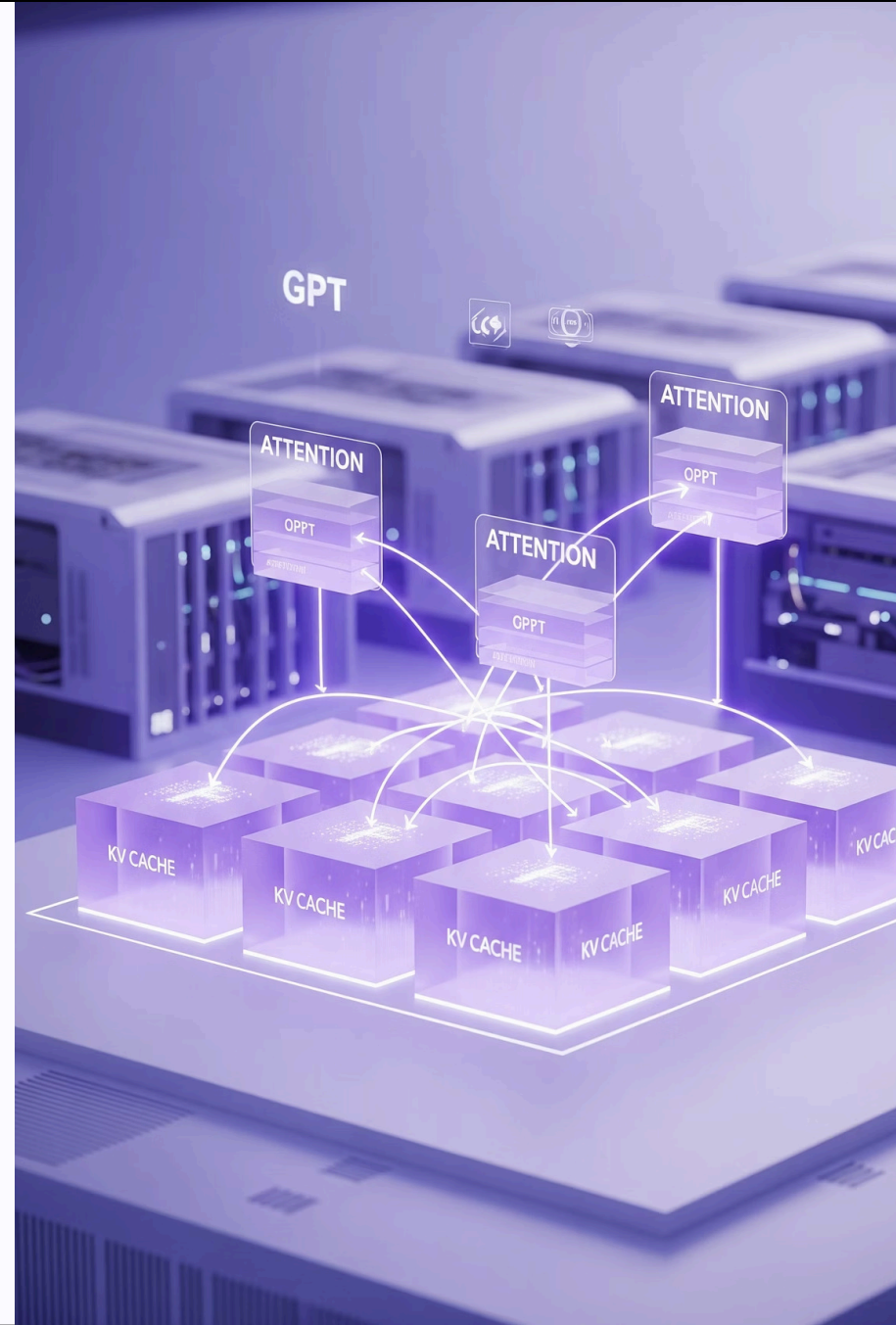
Long-sequence training introduces $O(n^2)$ memory costs, requiring specialized techniques

Inference Optimization

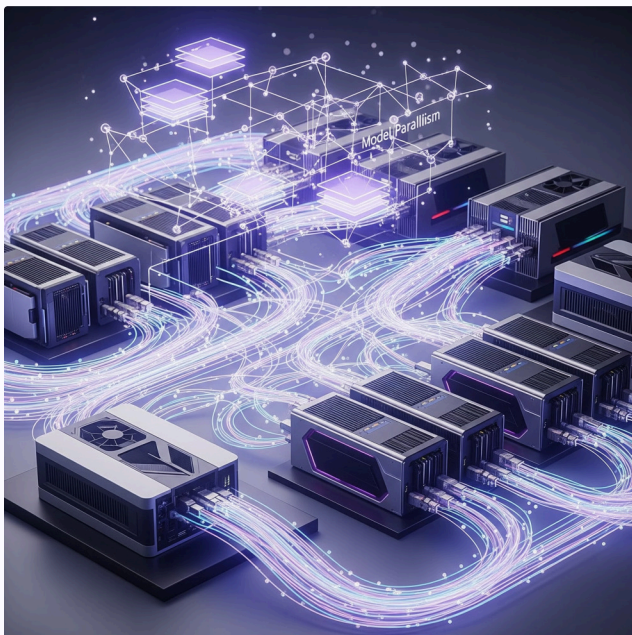
KV-cache implementation critical for incremental decoding and inference acceleration

Application Focus

Ideal for **chatbots**, **completion**, **summarization**, and creative generative tasks



Training Pipelines



Distributed Parallelism

Implement **data, model, and tensor parallelism** for multi-node training



Precision Optimization

Mixed precision (BF16/FP16) for 2-3x throughput with minimal accuracy loss



Memory Management

Gradient checkpointing to fit longer sequences in limited VRAM



Observability

Integrate logging & experiment tracking (MLflow, W&B) for reproducibility

Data Engineering for Transformers

Tokenization

1

Process TBs of text with Rust-based fast tokenizers (HuggingFace Tokenizers)

Optimize for tokenization throughput with multi-threading

Versioning

3

Maintain dataset versioning with DVC or custom solutions

Track data lineage for reproducibility and compliance

2

Data Cleaning

Deduplicate at document and n-gram levels to prevent memorization

Filter low-quality web data using classifier-based approaches

4

I/O Optimization

Stream sharded data from distributed storage to avoid bottlenecks

Pre-tokenize and cache common datasets for repeated experiments



Serving BERT Models

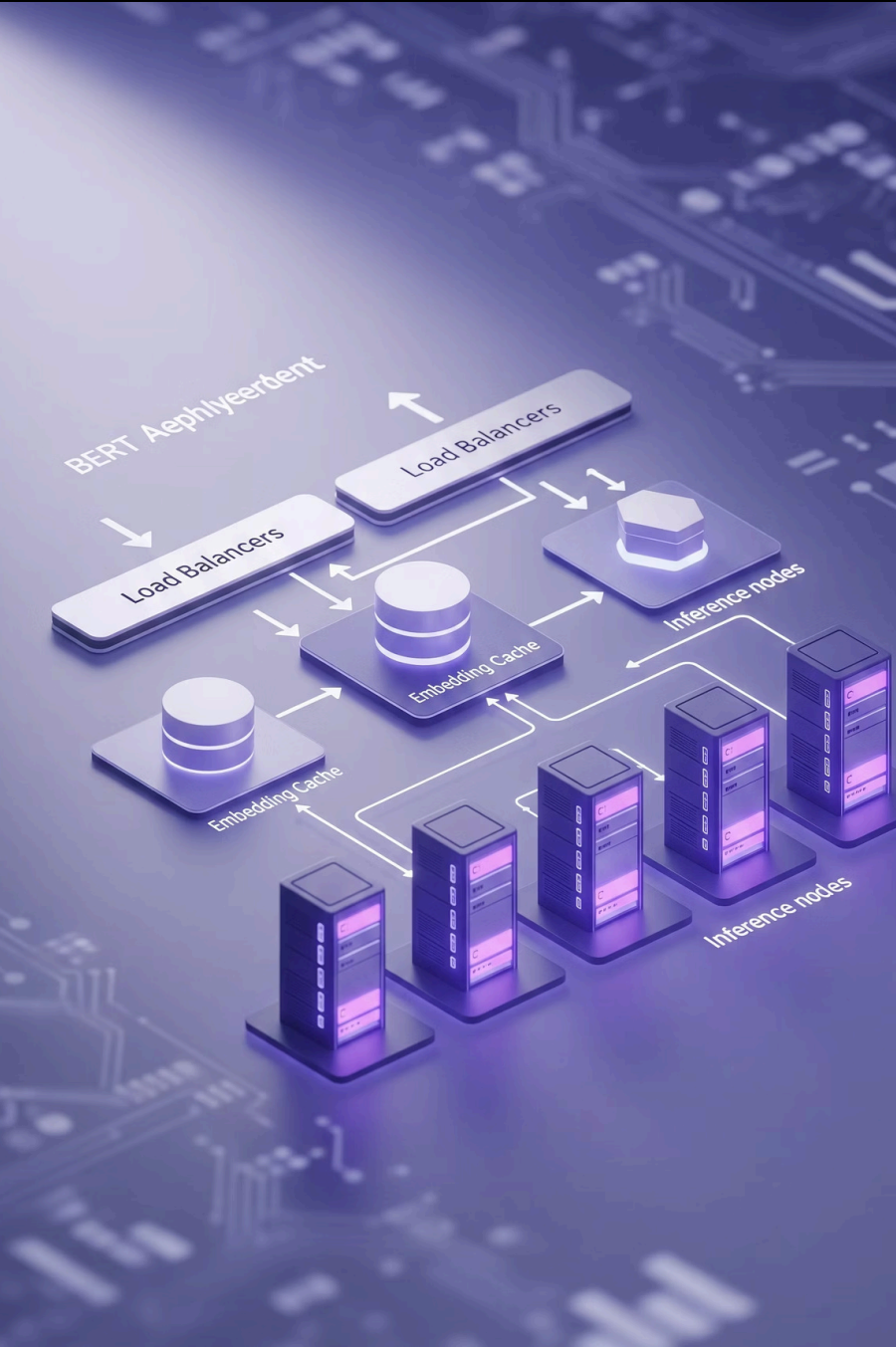
BERT models typically power [latency-sensitive embedding generation](#) for search, recommendation, and classification systems.

Optimization Techniques

- Deploy via ONNX Runtime or TensorRT for 3-5x speedup
- INT8 quantization for 2-4x throughput improvement
- Batch queries intelligently for higher GPU utilization

Deployment Architecture

- Scale with Kubernetes + HPA based on GPU utilization
- Implement embedding cache for frequent queries
- Monitor latency p50/p95/p99 as critical SLIs



Serving GPT Models

GPT inference requires specialized infrastructure to achieve **low latency** **for interactive chat** while managing significant computational demands.

KV-Cache Management

Reuse computation across tokens to reduce per-token cost by 65%+

Request Optimization

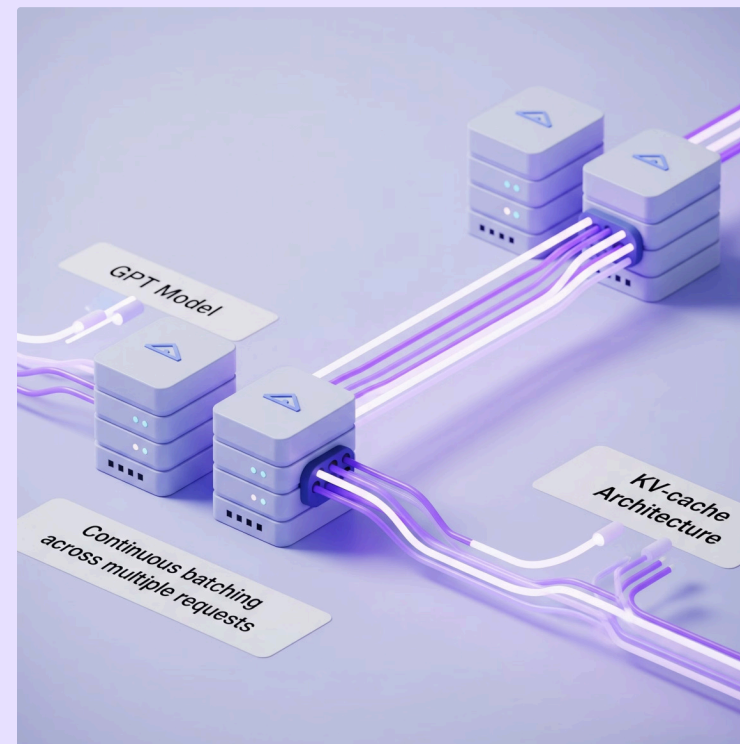
Implement prompt/response caching for popular queries

Deployment Options

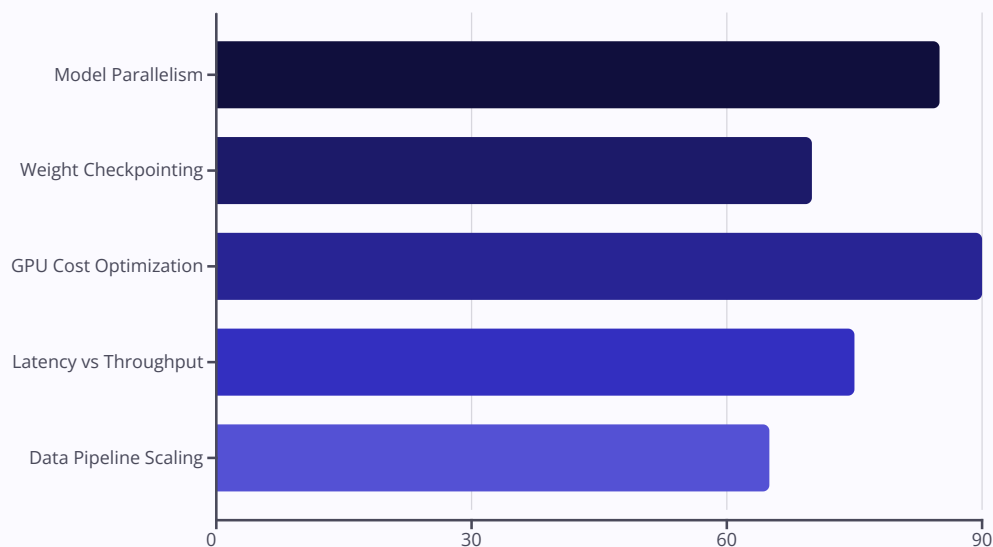
Leverage specialized servers: Triton, vLLM, or DeepSpeed-Inference

Continuous Batching

Dynamic scheduling to maximize throughput without latency spikes



Scaling Challenges



Core Infrastructure Challenges

- **Scale:** Model parallelism across hundreds of GPUs for GPT-3-class models
- **Reliability:** Checkpointing terabytes of weights for fault tolerance
- **Economics:** High cost of cloud GPUs drives extensive optimization efforts
- **Performance:** Serving infrastructure must balance [throughput vs user latency](#)
- **Monitoring:** Comprehensive telemetry across distributed components

Best Practices

BERT Optimization

Apply distillation + quantization to reduce size by 75%+ while maintaining 95%+ of accuracy

GPT Optimization

Implement PEFT (LoRA, P-Tuning) for 99% parameter reduction during fine-tuning

Production Monitoring

Monitor embedding drift and generation quality with automated evaluation pipelines

Key Takeaways



Architecture-Specific Infrastructure

- BERT → understanding tasks, embedding serving
- GPT → generation tasks, cache-heavy serving



Shared Optimization Techniques

- Efficient **parallelism** for training
- Strategic caching for inference
- Quantization for deployment

The difference between research models and **enterprise-ready systems** lies in the infrastructure:

- Robust data pipelines with quality checks
- Comprehensive monitoring and observability
- Cost-efficient scaling strategies
- Latency-optimized deployment architecture

Your infrastructure decisions directly impact both model performance and business viability